



操作系统原理

Operating Systems Principles

陈鹏飞
计算机学院



中山大學

SUN YAT-SEN UNIVERSITY

计算机学院 (软件学院)

SCHOOL OF COMPUTER SCIENCE AND ENGINEERING



第五讲 多线程



目标

- 线程概念，了解进程与线程的区别；
- 线程的基本设计与实现；
- 掌握用户级和内核级线程的差异、线程模型；
- Pthreads、Windows和Java线程库API；
- 隐式线程
- 讨论多线程编程问题；
- Windows 、Linux等操作系统中的线程管理功能；



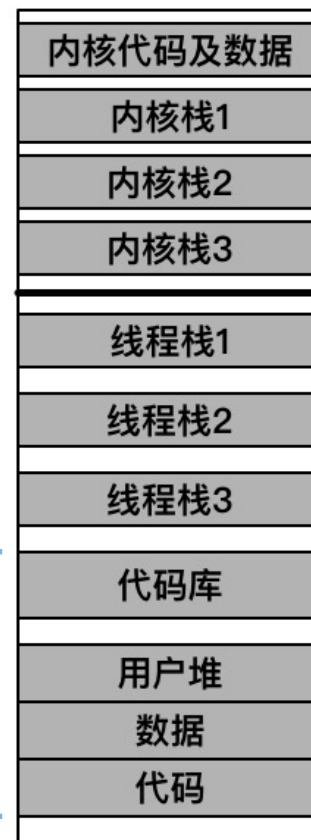
线程

- ❖ 线程是CPU调度的基本单元；
- ❖ 线程只包含运行时的状态
 - 静态部分由进程提供；
 - 包括了执行所需的最小状态（主要是寄存器和栈）；
- ❖ 一个进程可以包含多个线程
 - 每个线程共享同一地址空间（方便数据共享和交互）；
 - 允许进程内并行；
- ❖ 线程包含线程ID、程序计数器、寄存器和堆栈；
- ❖ 与其他线程共享：代码段、数据段和其他操作系统资源；



线程

- ❖ 一个进程的多线程可以在不同处理器上同时执行
 - 调度的基本单元由进程变为了线程
 - 每个线程都有状态
 - 上下文切换的单位变为了线程
- ❖ 每个线程拥有自己的栈;
- ❖ 内核中也有为线程准备的内核栈;
- ❖ 其它区域共享
 - 数据、代码、堆……



共享



进程与线程

进程的两个特点:

❖ 资源所有权

- 进程包括存放进程映像的虚拟机地址空间;
 - ✓ 操作系统提供预防进程间发生不必要资源冲突保护的功能。

❖ 调度/执行

- 进程执行时采用一个多程序的执行路径 (轨迹), 不同进程的执行过程会交替进行;
 - ✓ 进程具有执行态 (运行、就绪等) 和分配给其的优先级, 是可被操作系统调度和分派的实体;



进程与线程

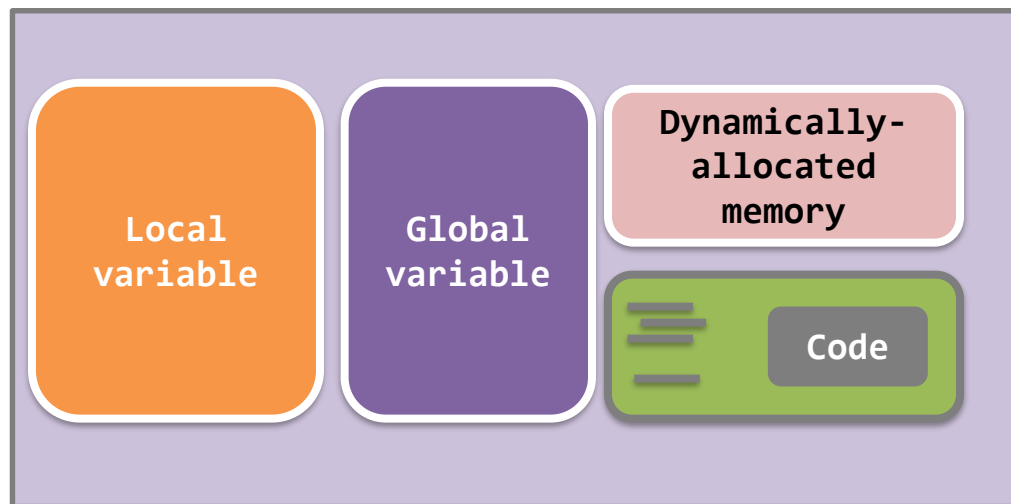
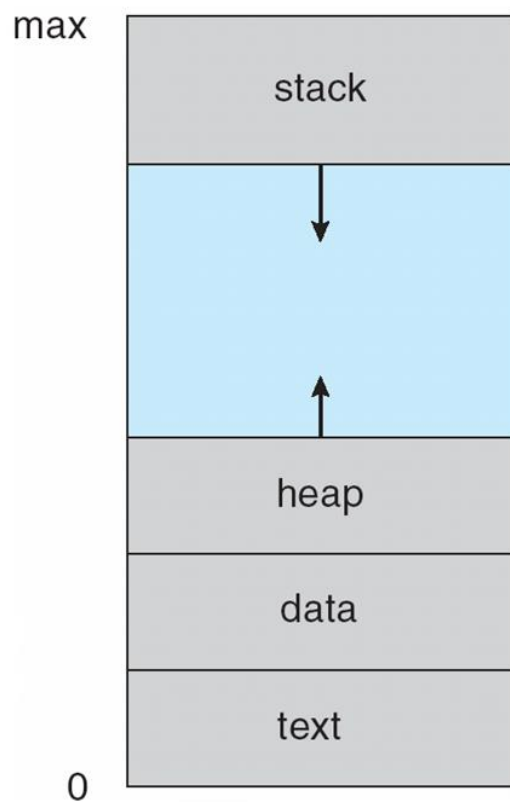
- ❖ 通常将分派的单位称为线程或轻量级进程（LWP）；
- ❖ 将拥有资源所有权的单位称为进程或任务；
- ❖ 多线程：

多线程是指操作系统在单个进程内支持多个并发执行路径的能力。



进程在内存中的映像

- User-space memory of Process A

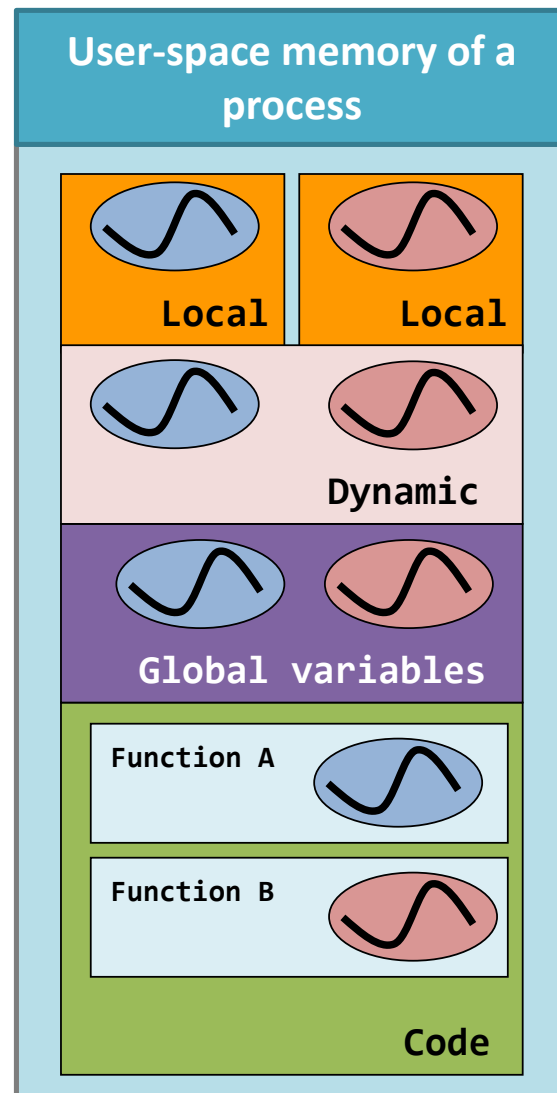




线程在内存中的分布

Code 代码部分

- All threads **share** the same code.
- A thread starts with **one specific function**.
 - We name it the **thread function**
 - Functions A & B in the diagram
- The thread function can invoke other functions or system calls
- But, a thread could never return to the caller of the thread function.



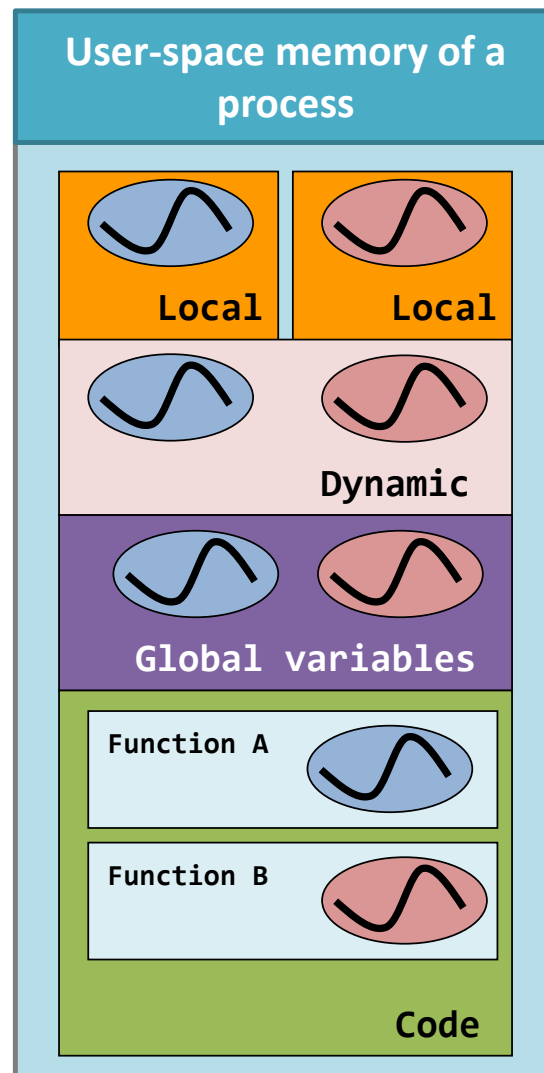


线程在内存中的分布

Dynamically allocated memory

Global variables

- All threads **share** the same global variable zone and the same dynamically allocated memory
- All threads can read from and write to both areas

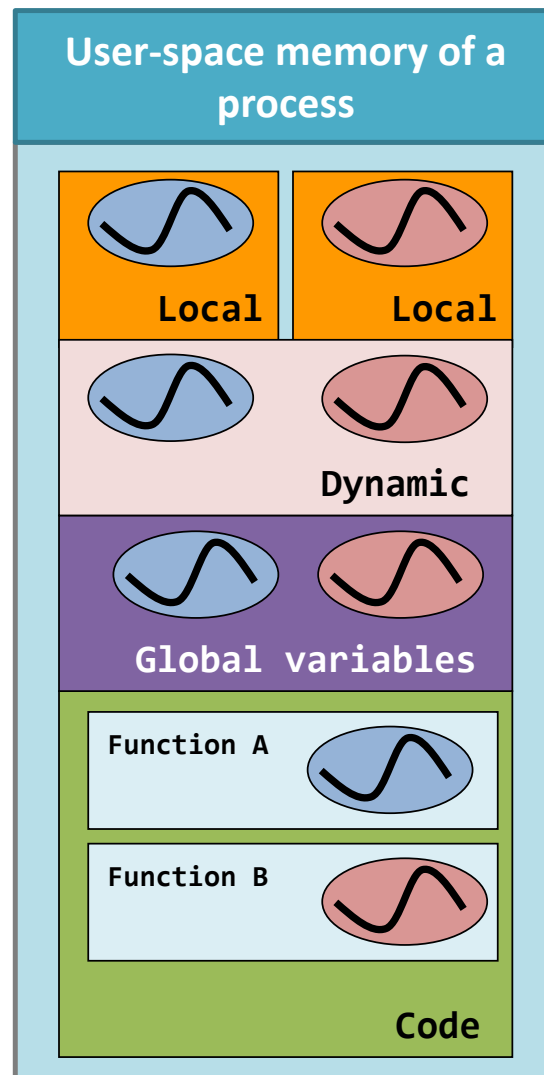




线程在内存中的分布

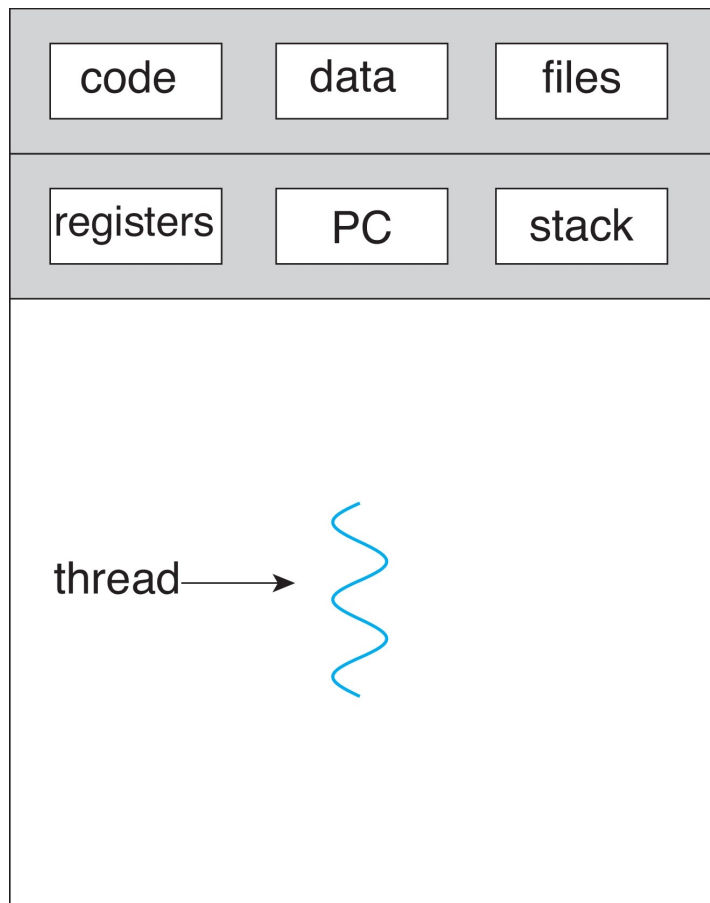
Local variables

- Each thread has its own memory range for the local variables
- So, the stack is the private zone for each stack

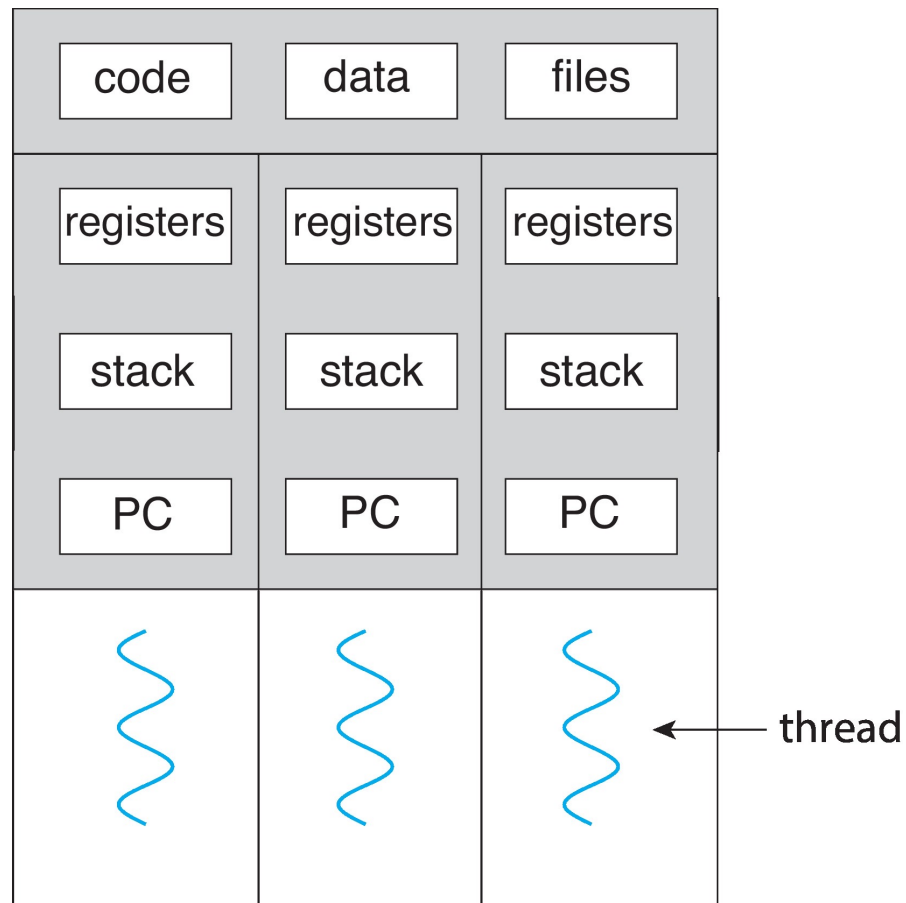




单线程进程和多线程进程



single-threaded process



multithreaded process

MS-DOS是支持单用户进程和单线程的操作系统例子。

Demo

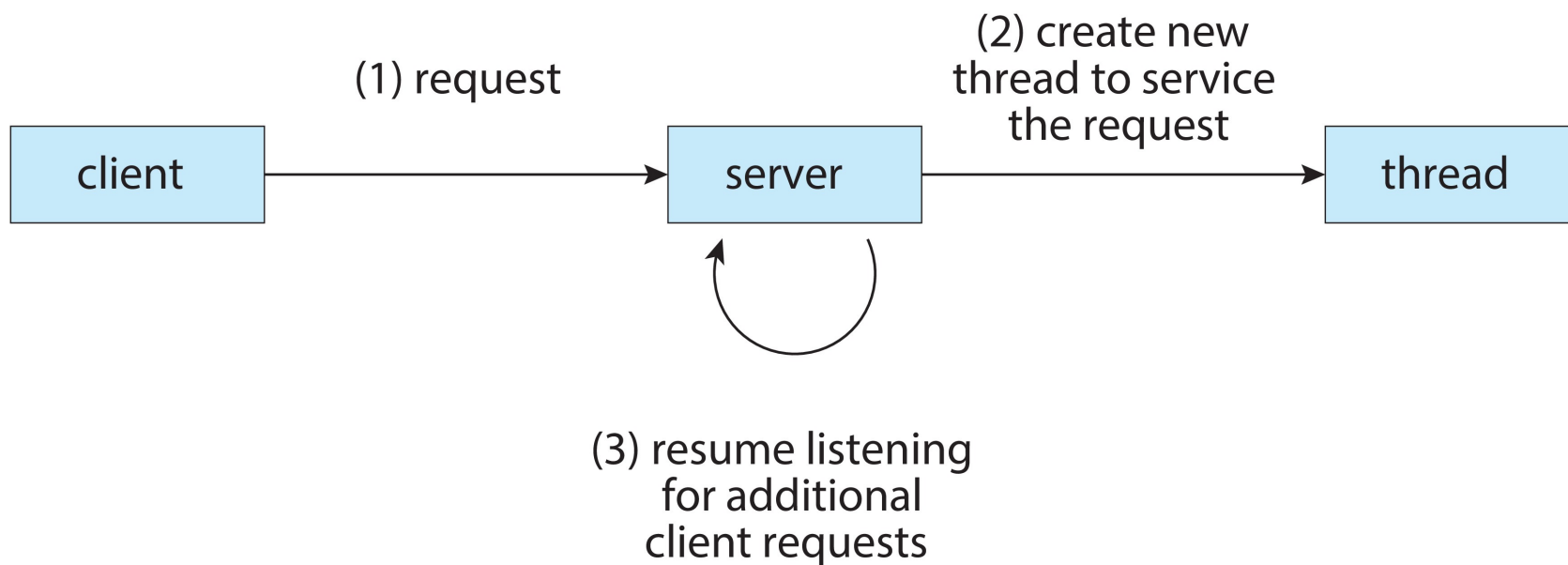


动机

- ❖ 大多数现代应用程序都是多线程的
- ❖ 线程在应用程序中运行；
- ❖ 应用程序的多个任务可以由单独的线程实现如Web浏览器
 - 更新显示
 - 获取数据
 - 拼写检查
 - 响应网络请求
- ❖ 进程创建很重载，而线程创建很轻量；
- ❖ 可以简化代码，提高效率；
- ❖ 内核通常是多线程的，多个线程在内核中运行，每个线程执行一个特定的任务；



动机

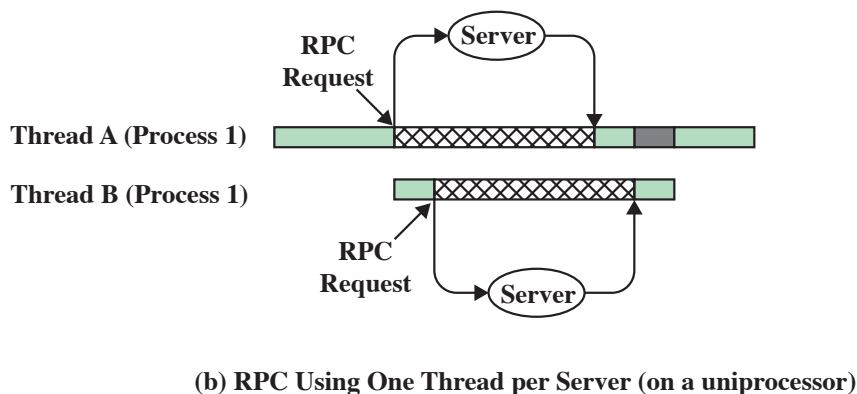
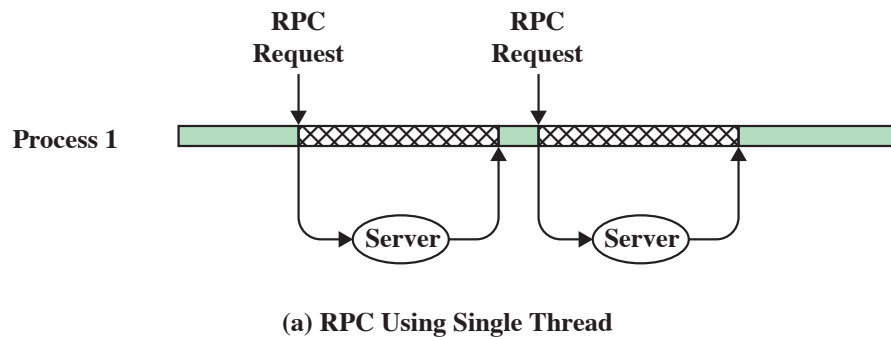


多线程服务器



动机

- 使用多线程的RPC;
允许服务处理多个并发请求;



-  Blocked, waiting for response to RPC
-  Blocked, waiting for processor, which is in use by Thread B
-  Running

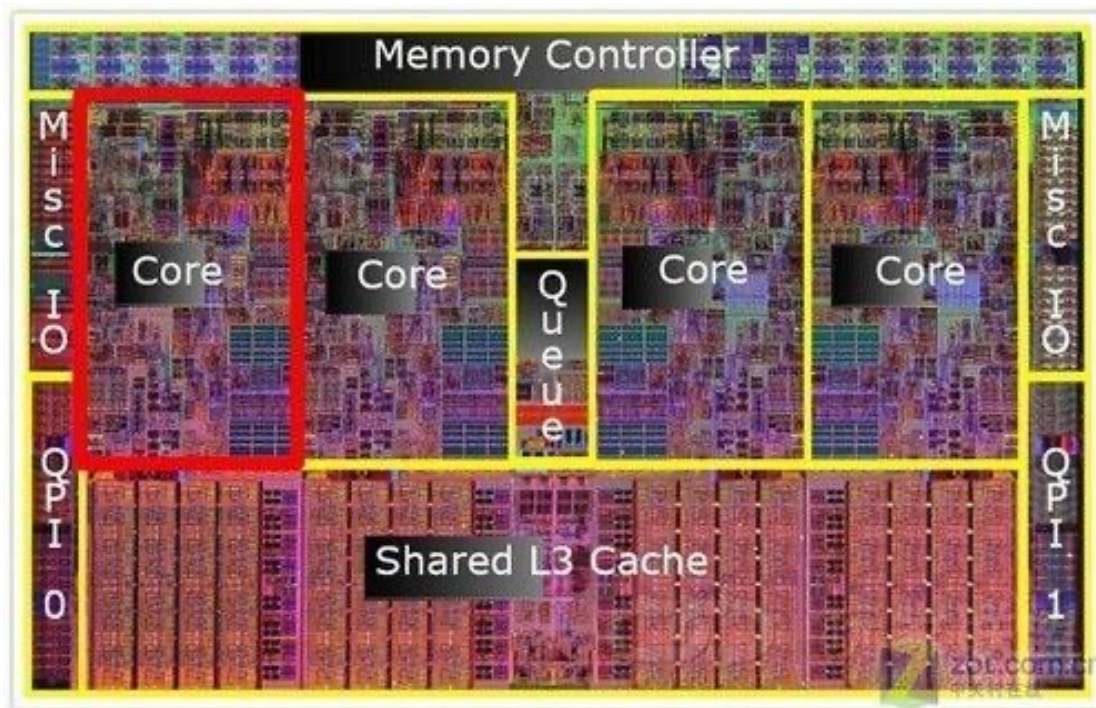
Figure 4.3 Remote Procedure Call (RPC) Using Threads



动机

➤ 从系统角度看：

1. 现代计算机通常包含多核；
2. 每个处理器一次只能运行一个进程；
3. CPU资源没有充分利用；
4. 为了提高效率，每个核心一个任务；





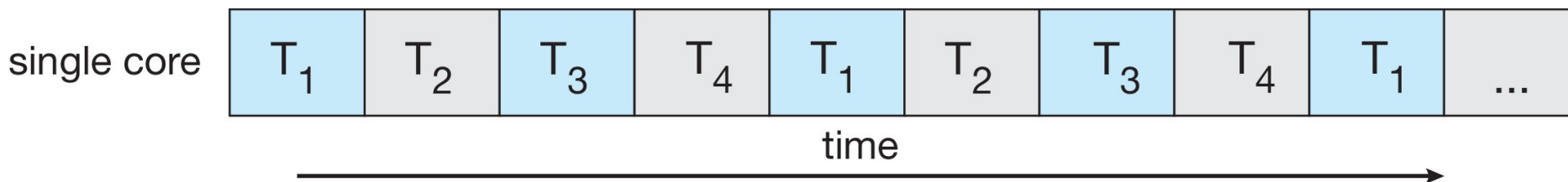
多线程的优点

- ❖ 响应性 - 如果部分线程被阻塞, 仍可以继续执行, 这对于用户界面设计尤其重要, 快速响应;
- ❖ 资源共享 - 线程共享进程资源, 比进程共享内存或消息传递更容易, 允许一个应用程序在同一地址空间内有多个不同的活跃线程;
- ❖ 经济 - 比进程创建更便宜, 线程切换比上下文切换开销更低。进程创建是线程创建的几十倍, 进程切换比线程切换慢5倍;
- ❖ 可扩展性 (可伸缩性) - 线程可以利用多核体系结构;

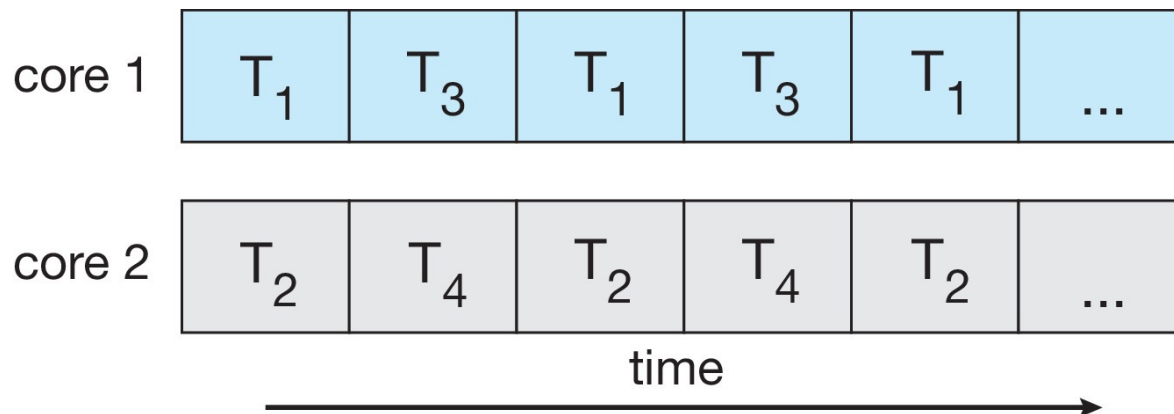


并发与并行

- 在单核系统上并发执行：通过快速的CPU调度进程，支持多个任务，提供并行的假象，任意时刻只能执行一个任务；



- 多核系统上的并行性：可以同时执行多个任务；





Amdahl I 定律 (公式)

- ❖ 对于同时具有串行和并行组件的应用程序，确定添加额外计算核心而获得的性能增益
- ❖ S是串行部分
- ❖ N处理核

$$speedup \leq \frac{1}{S + \frac{(1-S)}{N}}$$

- ❖ 也就是说，如果应用程序是75%并行/25%串行的，则从1个核增加到2个核会产生1.6倍的加速
- ❖ 当N接近无穷大时，加速比接近1/S

应用程序的串行部分对通过添加额外计算核心而获得的性能提升有着不成比例的影响

- ❖ 但定律是否考虑了当代多核系统？(或许不再使用)



多核编程

- ❖ 多核或多处理器系统给程序员带来压力，挑战包括：
 - 识别任务：分析应用程序，划分为独立、并发的任务；
 - 均衡：确保每个任务的执行量是相同的；
 - 数据分割：应用程序要成为独立任务，由任务访问和操作的数据也应划分以便运行在单独的核心上；
 - 数据依赖性：分析任务访问数据的依赖性，如果有数据依赖，确保任务执行是同步的，以适应依赖性；
 - 测试和调试：程序并行运行在多核上，执行路径有多条切不确定，给测试和调试带来困难；
- ❖ 并行性意味着系统可以同时执行多个任务；
- ❖ 并发支持多个正在进行的任务
 - 单处理器/内核，调度器提供并发性



多核编程

❖ 并行的类型

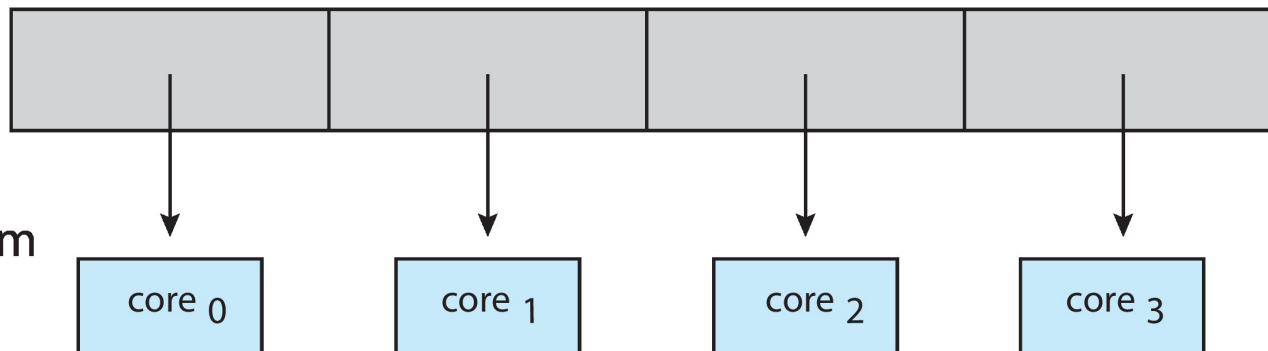
- 数据并行性 - 将相同数据的子集分布在多个核上，每个核上的操作相同；
- 任务并行性 - 将任务而不是数据分配到多个计算核心，每个线程执行唯一的操作，不同线程可以操作相同的数据，也可以操作不同的数据；
- 在大多情况下，难以实现严格的数据或者任务并行，混合使用两种策略；



数据并行和任务并行

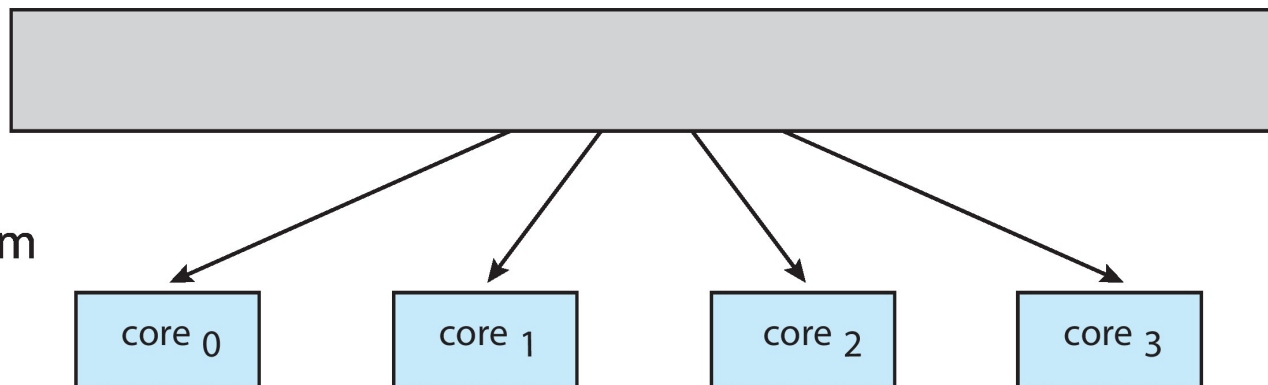
data

data
parallelism



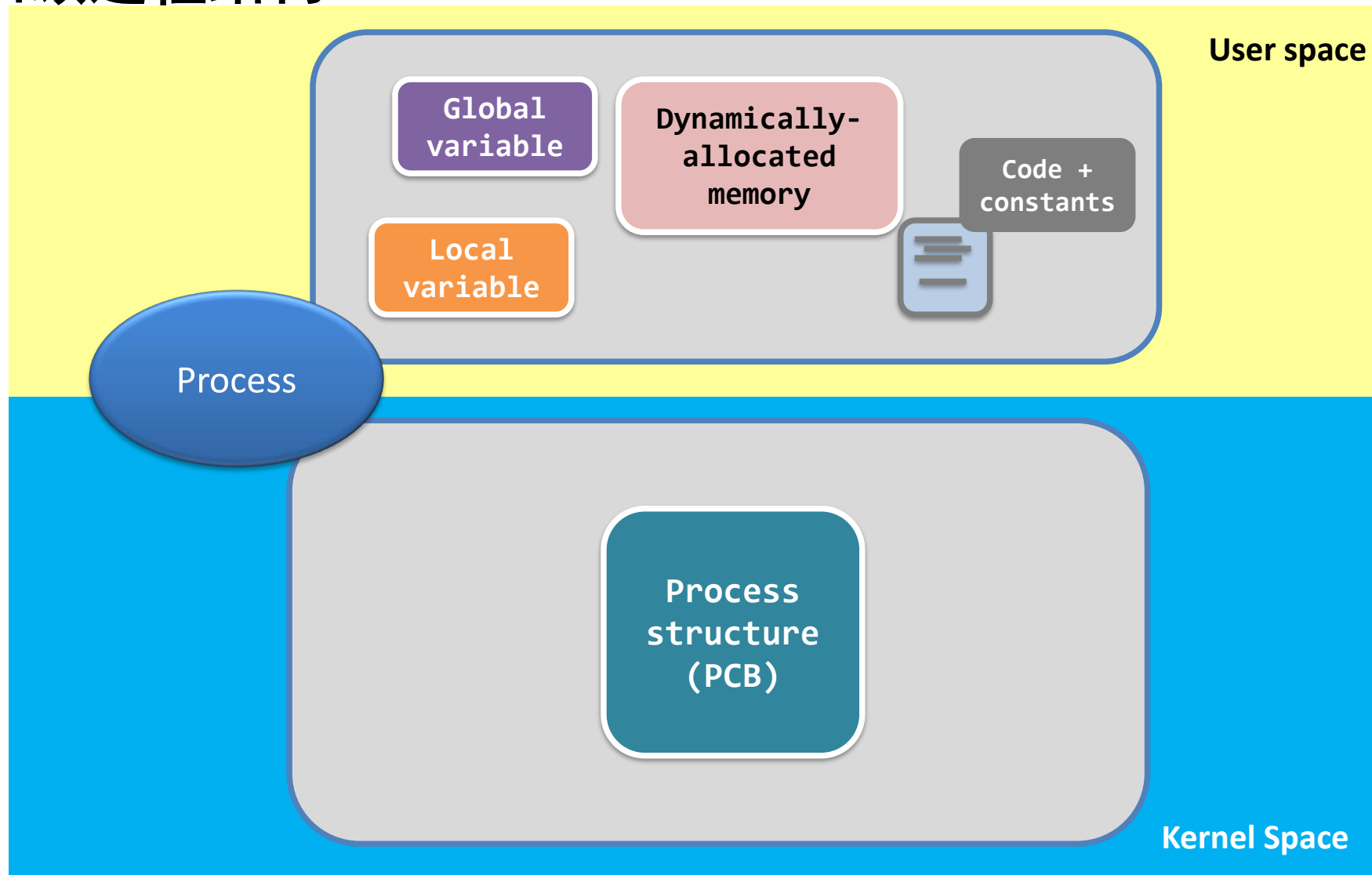
data

task
parallelism





回顾进程结构





线程控制块（TCB）

与PCB结构类似：

Linux中进程与线程使用的是同一种数据结构（task_struct），上下文切换中会使用

Thread ID
Thread state
CPU information : Program counter Register contents
Thread priority
Pointer to process that created this thread
Pointer(s) to other thread(s) that were created by this thread



如何管理线程？

- ❖ 如何控制和管理线程在用户空间的内存？
- ❖ 线程包括了用户空间的数据和资源，内核中的数据结构（TCB）；
- ❖ 线程分成用户线程和内核线程；
- ❖ 线程模型即用户线程与内核线程的对应关系；



中山大學

SUN YAT-SEN UNIVERSITY

计算机学院 (软件学院)

SCHOOL OF COMPUTER SCIENCE AND ENGINEERING



线程的类型

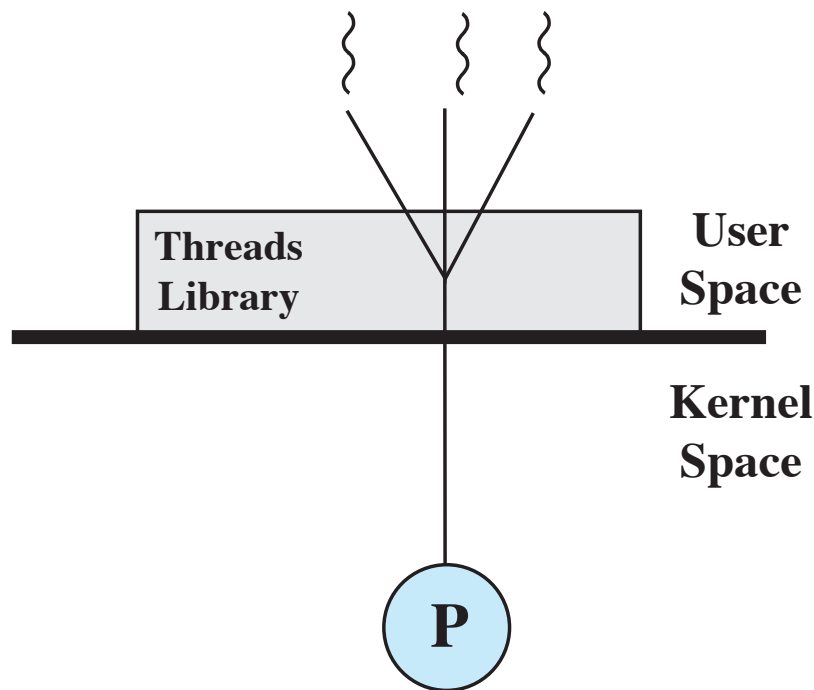
用户空间线程
(ULT)

内核空间线程 (KLT)



用户级线程

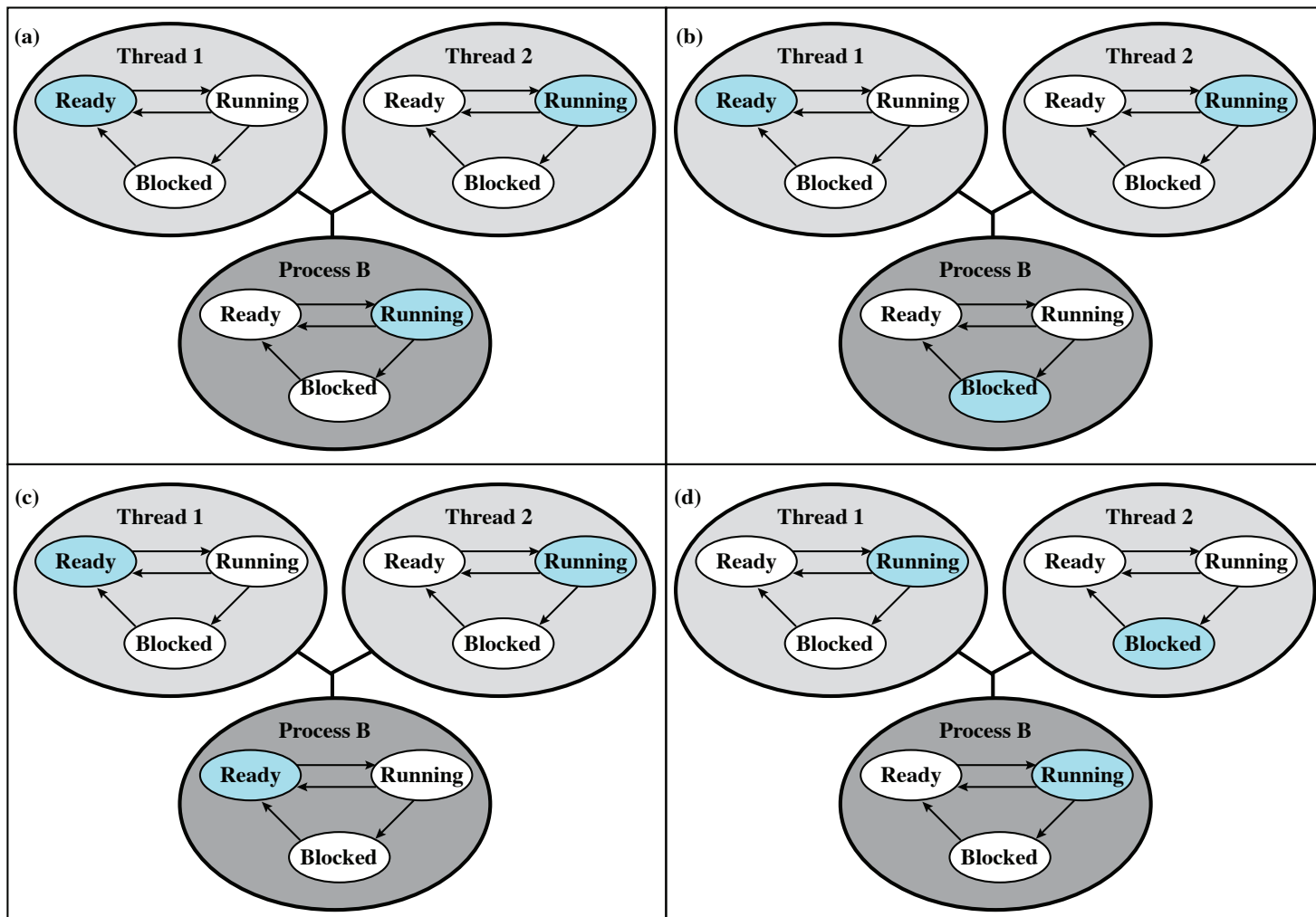
- ❖ 管理线程的所有工作都由应用程序完成;
- ❖ 内核意识不到线程的存在;
- ❖ 采用第三方库来创建如 pthreads;



(a) Pure user-level



用户级线程



Colored state
is current state

Figure 4.6 Examples of the Relationships Between User-Level Thread States and Process States

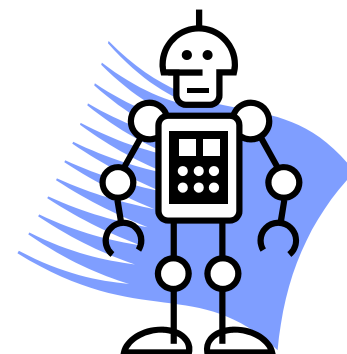


用户级线程的优势

Thread switching does not require kernel mode privileges

Scheduling can be application specific

ULTs can run on any OS





用户级线程的劣势

- ❖ 在典型的操作系统中，很多系统调用都会引起阻塞。因此，在ULT执行一个系统调用时，不仅会阻塞线程，也会阻塞进程中的所有线程。
- ❖ 在纯ULT策略中，多线程应用程序不能利用多处理技术。内核一次只把一个进程分配给一个处理器，因此一个进程中只有一个线程可以执行。





克服ULT的劣势

套管 (Jacketing)

- 把一个产生阻塞的系统调用转化为一个非阻塞的系统调用；

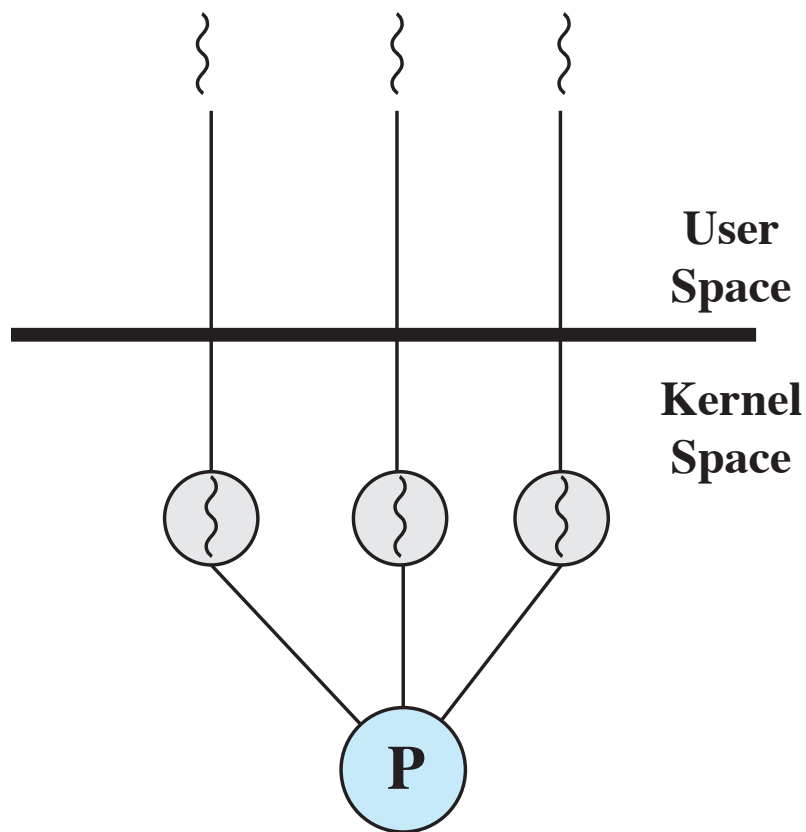


把应用程序写成一个多进程程序而非多线程程序，
每次切换都变成进程间的切换而非线程间切换





内核线程



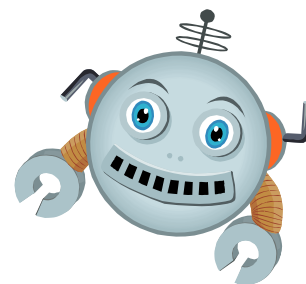
(b) Pure kernel-level

- 线程的管理工作由内核完成；
- 应用级没有线程管理代码，只有一个内核线程设施的应用编程接口（API）；
- Windows是这种方法的一个例子；



内核线程的优势

- ❖ 内核可以同时把同一个进程中的多个线程调度到多个处理器；
- ❖ 进程中的一个线程阻塞，内核可以调度同一个进程中的另一个线程；
- ❖ 内核线程自身也可以是多线程；





内核线程的劣势

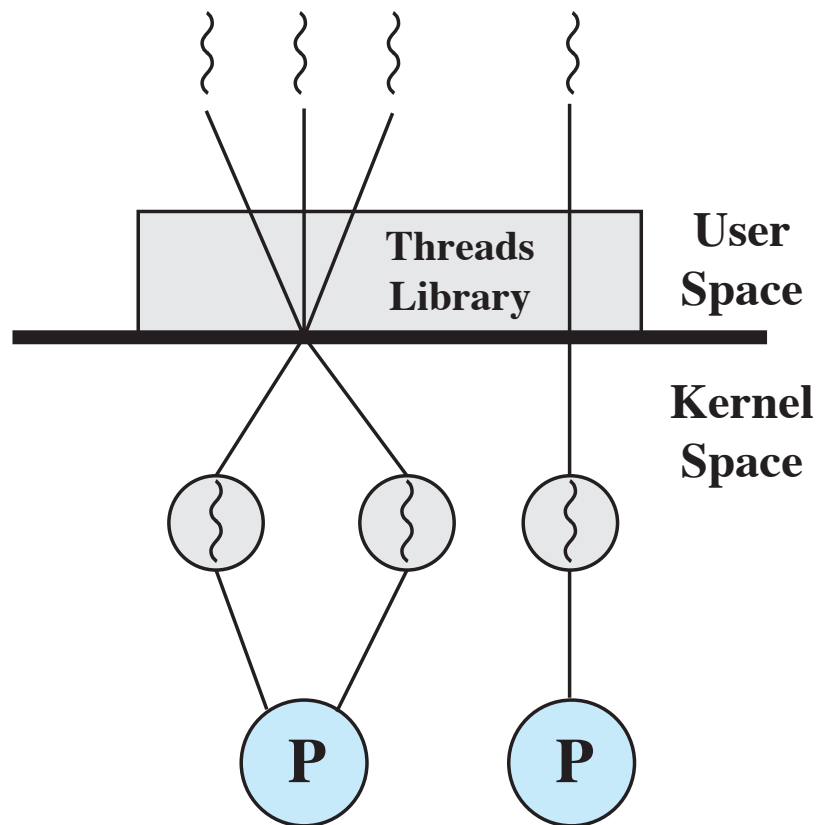
KLT的主要缺点：把控制权从一个线程传送到同一个进程内的另一个线程时，需要切换内核模式。

Operation	User-Level Threads	Kernel-Level Threads	Processes
Null Fork	34	948	11,300
Signal Wait	37	441	1,840



混合方法

- ❖ 线程的创建完全在用户空间中完成；
- ❖ 线程的调度和同步也在应用程序中进行。
- ❖ 多个用户线程会映射到一些内核线程上；
- ❖ Solaris 是一个例子；



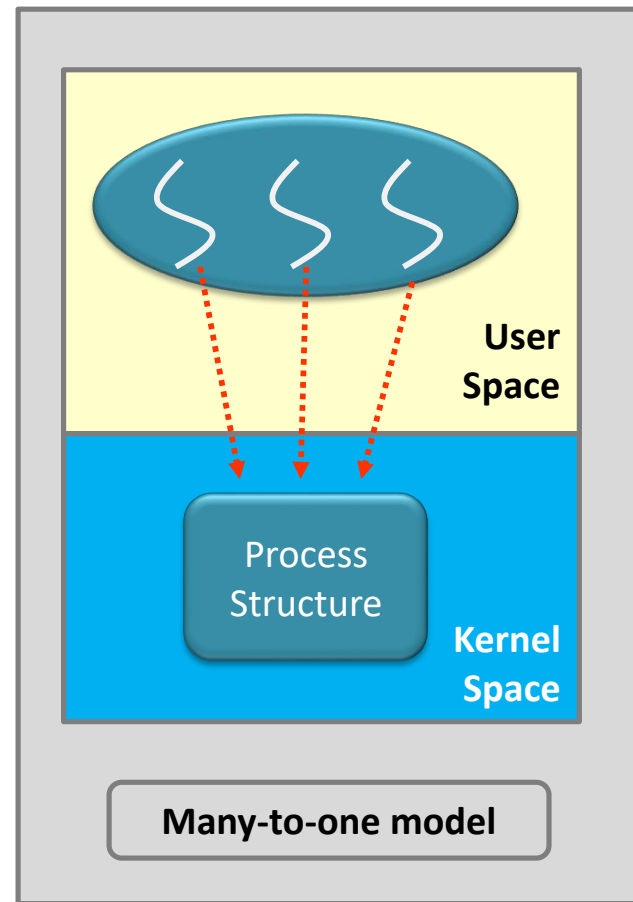
(c) Combined



用户线程和内核线程之间的关系——线程模型

• Many-to-One Model

- All the threads are mapped to one process structure in the kernel.
- Merit
 - Easy for the kernel to implement.
- Drawback
 - When a blocking system call is called, all the threads will be blocked
- Example. Old UNIX & green thread in some programming languages.

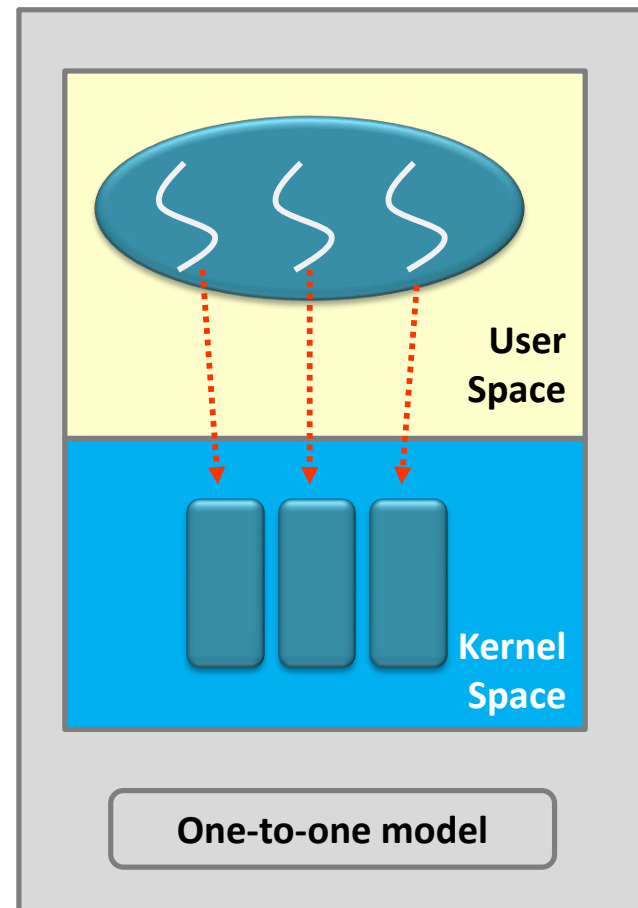




用户线程和内核线程之间的关系——线程模型

• One-to-One Model

- Each thread is mapped to a process or a thread structure
- Merit:
 - Calling blocking system calls only block those calling threads
 - A high degree of concurrency
- Drawback:
 - Cannot create too many threads as it is restricted by the size of the kernel memory
- Example. Linux and Windows follow this thread model

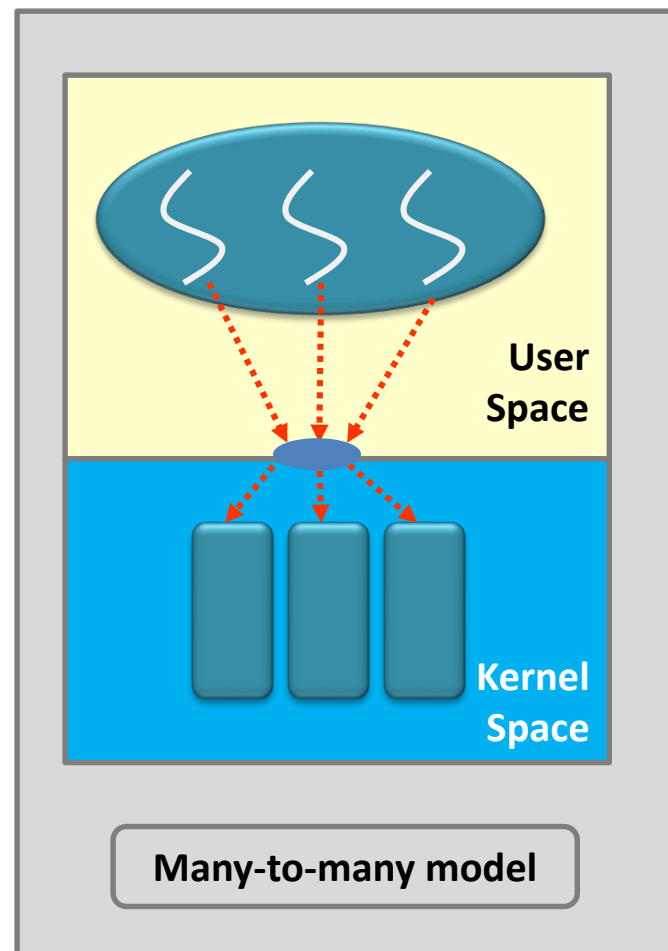




用户线程和内核线程之间的关系——线程模型

• Many-to-many Model

- Multiple threads are mapped to multiple structures (group mapping)
- Merit:
 - Create as many threads as necessary
 - Also have a high degree of concurrency





进程和线程之间的关系

Threads:Processes	Description	Example Systems
1:1	Each thread of execution is a unique process with its own address space and resources.	Traditional UNIX implementations
M:1	A process defines an address space and dynamic resource ownership. Multiple threads may be created and executed within that process.	Windows NT, Solaris, Linux, OS/2, OS/390, MACH
1:M	A thread may migrate from one process environment to another. This allows a thread to be easily moved among distinct systems.	Ra (Clouds), Emerald
M:N	Combines attributes of M:1 and 1:M cases.	TRIX

Table 4.2

Relationship between Threads and Processes



线程库

- ❖ 线程库为程序员提供了创建和管理线程的API;
- ❖ 两种主要的实施方式:
 - 第一种方式: 在用户空间中提供一个没有内核支持的库;
 - 第二种方法是实现操作系统支持的内核级库;
- ❖ 三种主要的线程库:
 - POSIX Pthreads (用户层次或者内核层次);
 - Windows (内核层次);
 - Java (利用windows API或者Pthreads来实现);



创建多线程

- Asynchronous threading 异步线程
 - Parent resumes execution after creating a child
 - Parent and child execute **concurrently**
 - Each thread runs independently
 - Little data sharing
- Synchronous threading 同步线程
 - Fork-join strategy: Parent waits for children to terminate
 - Significant data sharing



Pthreads

- ❖ 可以作为用户级或内核级提供;
- ❖ 用于线程创建和同步的POSIX标准 (IEEE 1003.1c) API 规范, 而不是实现;
- ❖ API指定了线程库的行为, 实现取决于库的开发;
- ❖ 在UNIX操作系统 (Linux和Mac OS X) 中常见;

	Process	Thread
Creation	<code>fork()</code>	<code>pthread_create()</code>
I.D. Type	PID, an integer	"pthread_t", a structure
Who am I?	<code>getpid()</code>	<code>pthread_self()</code>
Termination	<code>exit()</code>	<code>pthread_exit()</code>
Wait for child termination	<code>wait()</code> or <code>waitpid()</code>	<code>pthread_join()</code>
Kill?	<code>kill()</code>	<code>pthread_kill()</code>



Pthread线程创建

Thread creation – pthread_create()

Thread Function

```
1 void * hello( void *input ) {  
2     printf(“%s\n”, (char *) input);  
3     pthread_exit(NULL);  
4 }
```

Main Function

```
5 int main(void) {  
6     pthread_t tid;  
7     pthread_create(&tid, NULL, hello, “hello world”);  
8     pthread_join(tid, NULL);  
9     return 0;  
10 }
```



Pthread线程创建

Thread creation – pthread_create()

Thread Function

```
1 void * hello( void *input ) {  
2     printf(“%s\n”, (char *) input);  
3     pthread_exit(NULL);  
4 }
```

Main Function

```
5 int main(void) {  
6     pthread_t tid;  
7     pthread_create(&tid, NULL, hello, “hello world”);  
8     pthread_join(tid, NULL);  
9     return 0;  
10 }
```

At the beginning,
there is only one
thread running: **the
main thread.**



Main Thread



Pthread线程创建

Thread creation – pthread_create()

Thread Function

```
1 void * hello( void *input ) {  
2     printf("%s\n", (char *) input);  
3     pthread_exit(NULL);  
4 }
```

Main Function

```
5 int main(void) {  
6     pthread_t tid;  
7     pthread_create(&tid, NULL, hello, "hello world");  
8     pthread_join(tid, NULL);  
9     return 0;  
10 }
```

The hello thread is created!

It is running "together" with the main thread.



Main Thread

pthread_create()



Hello Thread



Pthread线程创建

Thread creation – pthread_create()

Thread Function

```
1 void * hello( void *input ) {  
2     printf("%s\n", (char *) input);  
3     pthread_exit(NULL);  
4 }
```

The **pthread_create()** function allows one argument to be passed to the thread function.

Main Function

```
5 int main(void) {  
6     pthread_t tid;  
7     pthread_create(&tid, NULL, hello, "hello world");  
8     pthread_join(tid, NULL);  
9     return 0;  
10 }
```

This sets the thread function of the to-be-created thread as: **hello()**.

Remember: A thread starts with **one specific function (thread function)**



Pthread线程创建

Thread creation – `pthread_create()`

Thread Function

```
1 void * hello( void *input ) {  
2     printf(“%s\n”, (char *) input);  
3     pthread_exit(NULL);  
4 }
```

Main Function

```
5 int main(void) {  
6     pthread_t tid;  
7     pthread_create(&tid, NULL, hello, “hello world”);  
8     pthread_join(tid, NULL);  
9     return 0;  
10 }
```

Remember `wait()`
and `waitpid()`?

`pthread_join()`
performs similarly.

Blocked

Main Thread

Hello Thread



Pthread线程创建

Thread creation – pthread_create()

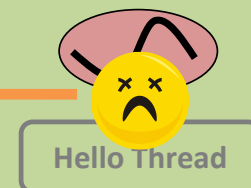
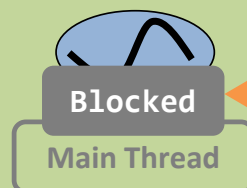
Thread Function

```
1 void * hello( void *input ) {  
2     printf(“%s\n”, (char *) input);  
3     pthread_exit(NULL);  
4 }
```

Main Function

```
5 int main(void) {  
6     pthread_t tid;  
7     pthread_create(&tid, NULL, hello, “hello world”);  
8     pthread_join(tid, NULL);  
9     return 0;  
10 }
```

Termination of the target thread causes pthread_join() to return.





Pthread线程创建——传递参数

Thread creation – passing parameter

Thread Function

```
1 void * do_your_job( void *input ) {  
2     printf("child = %d\n", *( (int *) input) );  
3     *((int *) input) = 20;  
4     printf("child = %d\n", *( (int *) input) );  
5     pthread_exit(NULL);  
6 }
```

Guess: What is the output?

Main Function

```
7 int main(void) {  
8     pthread_t tid;  
9     int input = 10;  
10    printf("main = %d\n", input);  
11    pthread_create(&tid, NULL, do_your_job, &input);  
12    pthread_join(tid, NULL);  
13    printf("main = %d\n", input);  
14    return 0;  
15 }
```

```
$ ./pthread_evil_1  
main = 10  
child = 10  
child = 20  
main = 20  
$
```



Each thread has a separated stack.

Why do we have such results?

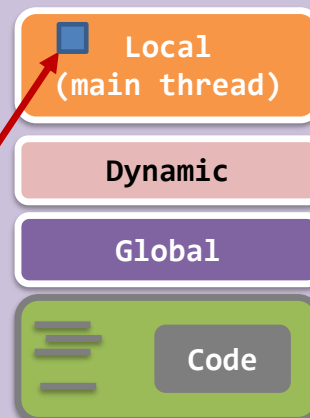


Pthread线程创建——传递参数

Thread creation – passing parameter

Well, we all know that the local variable “**input**” is in the stack for the main thread.

```
1 void * do_your_job( void *input ) {  
2     printf("child = %d\n", *( (int *) input) );  
3     *((int *) input) = 20;  
4     printf("child = %d\n", *( (int *) input) );  
5     pthread_exit(NULL);  
6 }  
  
7 int main(void) {  
8     pthread_t tid;  
9     int input = 10;  
10    printf("main = %d\n", input);  
11    pthread_create(&tid, NULL, do_your_job, &input);  
12    pthread_join(tid, NULL);  
13    printf("main = %d\n", input);  
13    return 0;  
14 }
```



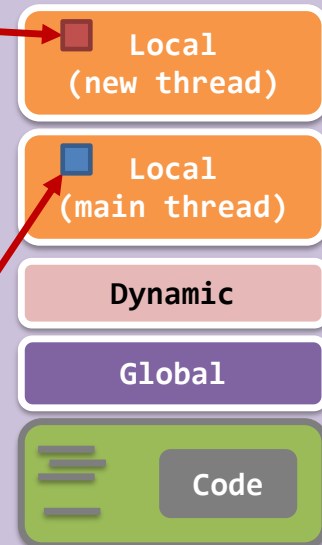


Pthread线程创建——传递参数

Thread creation – passing parameter

Yet...the stack for the new thread is not on another process, but is on the same piece of user-space memory as the main thread.

```
1 void * do_your_job( void *input ) {  
2     printf("child = %d\n", *( (int *) input) );  
3     *((int *) input) = 20;  
4     printf("child = %d\n", *( (int *) input) );  
5     pthread_exit(NULL);  
6 }  
  
7 int main(void) {  
8     pthread_t tid;  
9     int input = 10;  
10    printf("main = %d\n", input);  
11    pthread_create(&tid, NULL, do_your_job, &input);  
12    pthread_join(tid, NULL);  
13    printf("main = %d\n", input);  
13    return 0;  
14 }
```



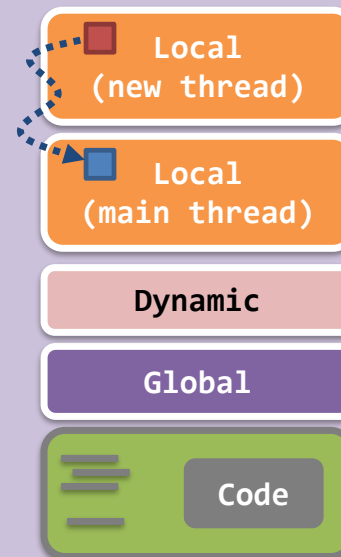


Pthread线程创建——传递参数

Thread creation – passing parameter

The `pthread_create()` function only passes an **address** to the new thread. Worse, the address is pointing to a variable in the stack of the main thread!

```
1 void * do_your_job( void *input ) {  
2     printf("child = %d\n", *( (int *) input) );  
3     *((int *) input) = 20;  
4     printf("child = %d\n", *( (int *) input) );  
5     pthread_exit(NULL);  
6 }  
  
7 int main(void) {  
8     pthread_t tid;  
9     int input = 10;  
10    printf("main = %d\n", input);  
11    pthread_create(&tid, NULL, do_your_job, &input);  
12    pthread_join(tid, NULL);  
13    printf("main = %d\n", input);  
13    return 0;  
14 }
```





Pthread线程创建——传递参数

Thread creation – passing parameter

Therefore, the new thread can change the value in the main thread, and vice versa.

```
1 void * do_your_job( void *input ) {  
2     printf("child = %d\n", *( (int *) input) );  
3     *((int *) input) = 20;  
4     printf("child = %d\n", *( (int *) input) );  
5     pthread_exit(NULL);  
6 }  
  
7 int main(void) {  
8     pthread_t tid;  
9     int input = 10;  
10    printf("main = %d\n", input);  
11    pthread_create(&tid, NULL, do_your_job, &input);  
12    pthread_join(tid, NULL);  
13    printf("main = %d\n, input);  
13    return 0;  
14 }
```

Local
(new thread)

Local
(main thread)

Dynamic

Global

Code



Pthread线程创建——多线程

Thread creation – multiple threads

Thread Function

```
1 void * do_your_job(void *input) {  
2     int id = *((int *) input);  
3     printf("My ID number = %d\n", id);  
4     pthread_exit(NULL);  
5 }
```

Main Function

```
6 int main(void) {  
7     int i;  
8     pthread_t tid[5];  
9  
10    for(i = 0; i < 5; i++)  
11        pthread_create(&tid[i], NULL, do_your_job, &i);  
12    for(i = 0; i < 5; i++)  
13        pthread_join(tid[i], NULL);  
14    return 0;  
15 }
```

Waiting on several threads: enclose pthread_join() within a for loop



Pthread线程销毁

Thread termination – passing return value

Thread Function

```
1 void * do_your_job(void *input) {  
2     int *output = (int *) malloc(sizeof(int));  
3     srand(time(NULL));  
4     *output = ((rand() % 10) + 1) * (*((int *) input));  
5     pthread_exit(output);  
6 }
```

`void pthread_exit(void *return_value);`

Together with termination, a pointer to a **global variable or a piece of dynamically allocated memory** is returned to the main thread.

Main Function

```
7 int main(void) {  
8     pthread_t tid;  
9     int input = 10, *output;  
10    pthread_create(&tid, NULL, do_your_job, &input);  
11    pthread_join(tid, (void **) &output);  
12    return 0;  
13 }
```

Using pass-by-reference, a pointer to the result is received in the main thread.



Windows多线程C程序

```
#include <windows.h>
#include <stdio.h>
DWORD Sum; /* data is shared by the thread(s) */

/* The thread will execute in this function */
DWORD WINAPI Summation(LPVOID Param)
{
    DWORD Upper = *(DWORD*)Param;
    for (DWORD i = 1; i <= Upper; i++)
        Sum += i;
    return 0;
}
```



Windows多线程C程序

```
int main(int argc, char *argv[])
{
    DWORD ThreadId;
    HANDLE ThreadHandle;
    int Param;

    Param = atoi(argv[1]);
    /* create the thread */
    ThreadHandle = CreateThread(
        NULL, /* default security attributes */
        0, /* default stack size */
        Summation, /* thread function */
        &Param, /* parameter to thread function */
        0, /* default creation flags */
        &ThreadId); /* returns the thread identifier */

    /* now wait for the thread to finish */
    WaitForSingleObject(ThreadHandle,INFINITE);

    /* close the thread handle */
    CloseHandle(ThreadHandle);

    printf("sum = %d\n",Sum);
}
```




Java线程

- ❖ Java线程由JVM管理;
- ❖ 通常使用底层操作系统提供的线程模型实现;
- ❖ Java线程可以通过以下方式创建: ;
 - 扩展线程类
 - 实现可运行接口
 - 标准实践是实现可运行接口

```
public interface Runnable
{
    public abstract void run();
}
```




Java线程

实现可运行接口:

```
class Task implements Runnable
{
    public void run() {
        System.out.println("I am a thread.");
    }
}
```

创建线程:

```
Thread worker = new Thread(new Task());
worker.start();
```

正在等待线程:

```
try {
    worker.join();
}
catch (InterruptedException ie) { }
```



Java线程

- ❖ Java还允许围绕Executor接口创建线程，而不是显式创建线程：
- ❖ Executor的用途如下：

```
public interface Executor
{
    void execute(Runnable command);
}
```

```
Executor service = new Executor();
service.execute(new Task());
```



Java执行器框架

```
import java.util.concurrent.*;

class Summation implements Callable<Integer>
{
    private int upper;
    public Summation(int upper) {
        this.upper = upper;
    }

    /* The thread will execute in this method */
    public Integer call() {
        int sum = 0;
        for (int i = 1; i <= upper; i++)
            sum += i;

        return new Integer(sum);
    }
}
```



Java执行器框架

```
public class Driver
{
    public static void main(String[] args) {
        int upper = Integer.parseInt(args[0]);

        ExecutorService pool = Executors.newSingleThreadExecutor();
        Future<Integer> result = pool.submit(new Summation(upper));

        try {
            System.out.println("sum = " + result.get());
        } catch (InterruptedException | ExecutionException ie) { }
    }
}
```



隐式多线程

- ❖ 随着线程数量的增加，程序的正确性变得越来越困难
- ❖ 由编译器和运行时库而不是程序员创建和管理线程，隐式多线程
- ❖ 探索了五种方法
 - 线程池
 - Fork-Join
 - OpenMP
 - 中央大调度 (GCD)
 - 英特尔线程构件 (TBB)



线程池

- ❖ 在池中创建多个线程，在其中等待工作
- ❖ 优势：
 - 使用现有线程处理请求通常比创建新线程快一点；
 - 允许将应用程序中的线程数限制到池的大小；
 - 将要执行的任务和创建任务的机制分离，可以使用不同的策略来运行任务
 - 例如，可以安排任务定期运行
- ❖ Windows API支持线程池：

```
DWORD WINAPI PoolFunction(AVOID Param) {  
    /*  
     * this function runs as a separate thread.  
     */  
}
```




线程池

❖ 在Executors类中创建线程池的三种工厂方法:

- `static ExecutorService newSingleThreadExecutor()`
- `static ExecutorService newFixedThreadPool(int size)`
- `static ExecutorService newCachedThreadPool()`

```
import java.util.concurrent.*;

public class ThreadPoolExample
{
    public static void main(String[] args) {
        int numTasks = Integer.parseInt(args[0].trim());

        /* Create the thread pool */
        ExecutorService pool = Executors.newCachedThreadPool();

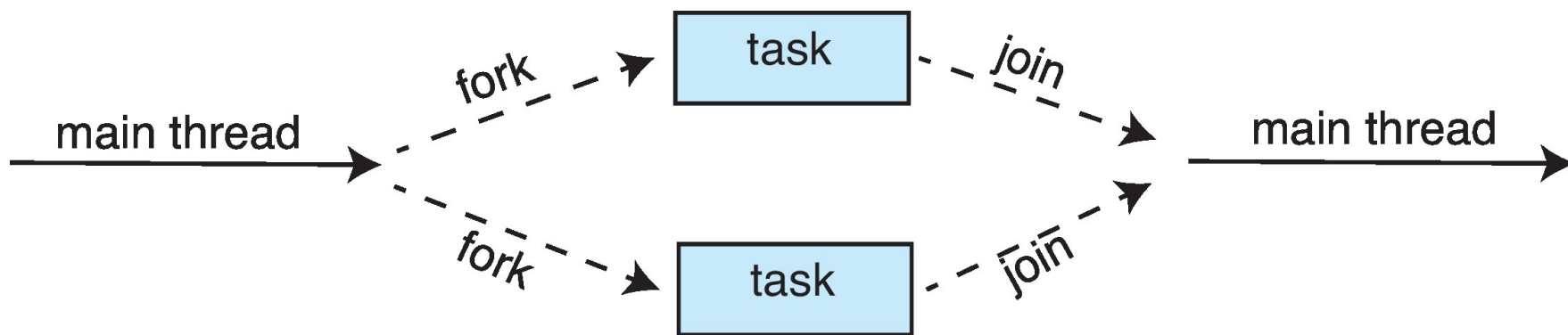
        /* Run each task using a thread in the pool */
        for (int i = 0; i < numTasks; i++)
            pool.execute(new Task());

        /* Shut down the pool once all threads have completed */
        pool.shutdown();
    }
}
```



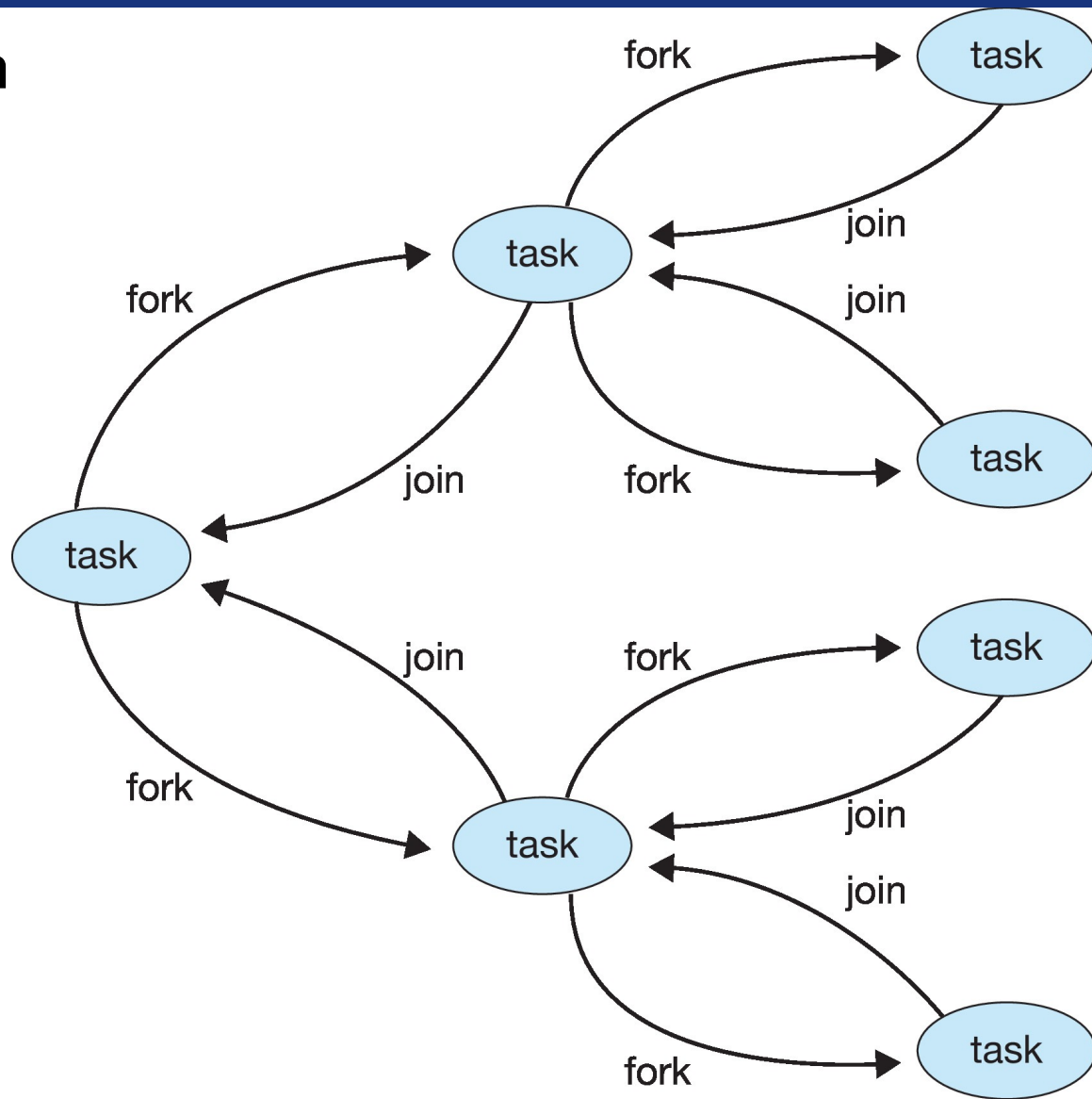
Fork-join

- ❖ 多个线程（任务）被分叉，然后连接。





Fork-join





OpenMP

- ❖ 编译器指令集和C、C++、FORTRAN的API
- ❖ 为共享内存环境中的并行编程提供支持
- ❖ 标识并行区域 - 可以并行运行的代码块

#pragma-omp并行

创建尽可能多的线程

```
#include <omp.h>
#include <stdio.h>

int main(int argc, char *argv[])
{
    /* sequential code */

    #pragma omp parallel
    {
        printf("I am a parallel region.");
    }

    /* sequential code */

    return 0;
}
```

❖ 并行运行for循环

```
#pragma omp parallel for
for (i = 0; i < N; i++) {
    c[i] = a[i] + b[i];
}
```



大中央调度

- ❖ 苹果macOS和iOS操作系统技术
- ❖ C、C++和Obj-C语言、API和运行库的扩展
- ❖ 允许识别并行部分
- ❖ 管理线程的大部分细节
- ❖ 块位于“`^{}:`”：`^ {printf ( “I am a block” ) ; }`
- ❖ 放置在调度队列中的块
 - 从队列中删除时分配给线程池中的可用线程



大中央调度

❖ 两种类型的调度队列:

- 串行 - 按FIFO顺序删除块, 队列为每个进程, 称为主队列
 - 程序员可以在程序中创建额外的串行队列
- 并发-按FIFO顺序删除, 但一次可以删除多个
 - 按服务质量划分的四个全系统队列:
 - QOS\u类\u用户\u交互
 - QOS\u类\u用户\u已启动
 - QOS\u类\u用户\u实用程序
 - QOS\u类\u用户\u后台



Intel TBB

- ❖ 设计并行C++程序的模板库
- ❖ 简单for循环的串行版本

```
for (int i = 0; i < n; i++) {  
    apply(v[i]);  
}
```

- ❖ 使用TBB和并行_for语句编写的循环也一样:

```
parallel_for (size_t(0), n, [=](size_t i) {apply(v[i]);});
```



多线程问题

- ❖ fork () 和 exec () 系统调用的语义
- ❖ 信号处理
 - 同步与异步
- ❖ 目标线程的线程取消
 - 异步还是延迟
- ❖ 线程本地存储
- ❖ 调度程序激活



Fork() 和 exec() 的语义

- ❖ fork() 是只复制调用线程还是复制所有线程?
 - 有些unix有两种fork版本
- ❖ exec() 通常正常工作 - 替换正在运行的进程，包括所有线程



信号处理

- ❖ 在UNIX系统中，信号用于通知进程某个特定事件已发生。
- ❖ 信号处理器用于处理信号
 1. 信号由特定事件产生
 2. 信号被传送到进程
 3. 信号由两个信号处理程序之一处理：
 1. 违约
 2. 用户定义
- ❖ 每个信号都有内核在处理信号时运行的默认处理程序
 - 用户定义的信号处理程序可以覆盖默认值
 - 对于单线程，信号传输到进程



信号处理

❖ 多线程的信号应该在哪里传递？

- 将信号传递到应用该信号的线程
- 将信号传递给进程中的每个线程
- 将信号传递给进程中的某些线程
- 指定一个特定线程来接收进程的所有信号



线程取消

- ❖ 在线程结束之前终止它
- ❖ 要取消的线程是目标线程
- ❖ 两种一般方法:
 - 异步取消立即终止目标线程
 - 延迟取消允许目标线程定期检查是否应该取消
- ❖ Pthread用于创建和取消线程的代码:

```
pthread_t tid;

/* create the thread */
pthread_create(&tid, 0, worker, NULL);

. . .

/* cancel the thread */
pthread_cancel(tid);

/* wait for the thread to terminate */
pthread_join(tid, NULL);
```




线程取消

- ❖ 调用线程取消请求取消，但实际取消取决于线程状态

Mode	State	Type
Off	Disabled	—
Deferred	Enabled	Deferred
Asynchronous	Enabled	Asynchronous

- ❖ 若线程已禁用取消，则取消将保持挂起状态，直到线程启用它为止
- ❖ 默认类型是延迟的
 - 取消仅在线程到达取消点时发生
 - i、 例如，`pthread_testcancel()`
 - 然后调用清理处理程序
- ❖ 在Linux系统上，线程取消是通过信号处理的



Java中的线程取消

- ❖ 延迟取消使用interrupt () 方法，该方法设置线程的中断状态。
 - 。
- ❖ 然后，线程可以检查它是否被中断：

```
Thread worker;
```

```
. . .
```

```
/* set the interruption status of the thread */  
worker.interrupt()
```

```
while (!Thread.currentThread().isInterrupted()) {  
    . . .  
}
```



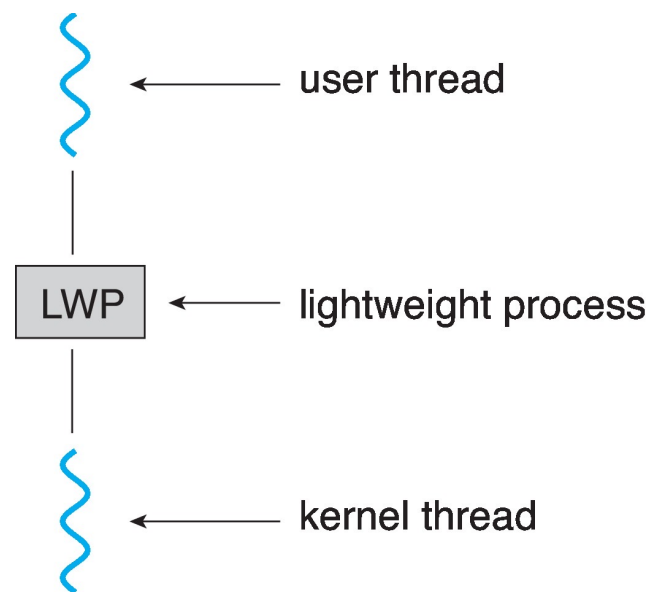
Java中的线程取消

- ❖ 线程本地存储 (TLS) 允许每个线程拥有自己的数据副本
- ❖ 当您无法控制线程创建过程时 (即, 当使用线程池时), 此功能非常有用
- ❖ 与局部变量不同
 - 局部变量仅在单个函数调用期间可见
 - TLS在函数调用中可见
- ❖ 类似于静态数据
 - TLS对于每个线程都是唯一的



调度程序激活

- ❖ M:M和两级模型都需要通信来维护分配给应用程序的适当数量的内核线程
- ❖ 通常在用户线程和内核线程之间使用中间数据结构 - 轻量级进程 (LWP)
 - 似乎是一个虚拟处理器，进程可以在该处理器上调度用户线程运行
 - 每个LWP都连接到内核线程
 - 要创建多少LWP?
- ❖ 调度器激活提供了upcall——一种从内核到线程库中upcall处理程序的通信机制
- ❖ 这种通信允许应用程序保持正确的内核线程数





Windows线程

- ❖ Windows API - Windows应用程序的主要API
- ❖ 实现内核级的一对一映射
- ❖ 每个线程包含
 - 线程id
 - 表示处理器状态的寄存器集
 - 当线程在用户模式或内核模式下运行时，分开用户和内核堆栈
 - 运行时库和动态链接库（DLL）使用的专用数据存储区域
- ❖ 寄存器集、堆栈和专用存储区域称为线程的上下文



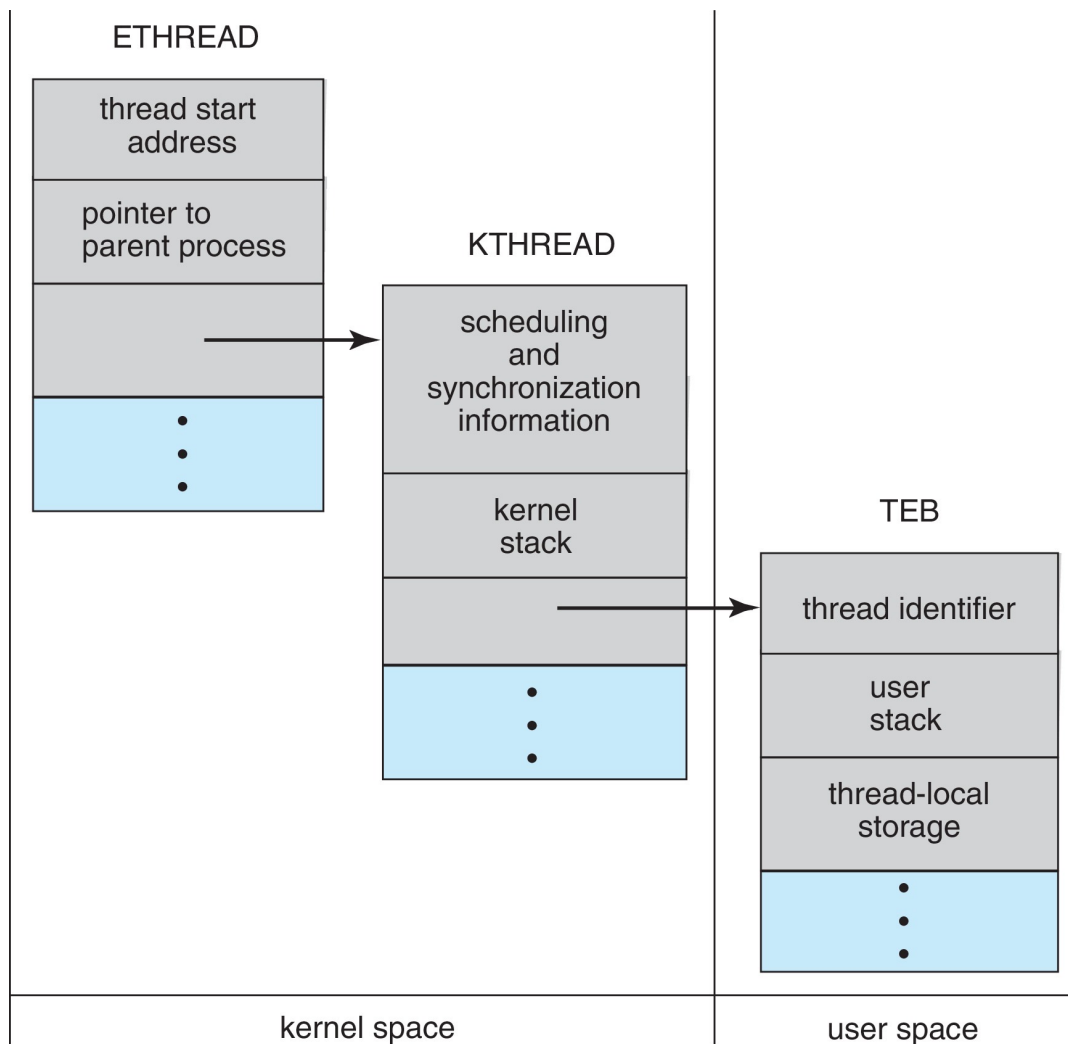
Windows线程

❖ 线程的主要数据结构包括:

- ETHREAD (执行线程块) - 在内核空间中包含指向线程所属进程和KTHREAD的指针
- KTHREAD (内核线程块) - 调度和同步信息, 内核模式堆栈, 指向TEB的指针, 在内核空间中
- TEB (线程环境块) - 用户空间中的线程id、用户模式堆栈、线程本地存储



Windows线程数据结构





Linux线程

- ❖ Linux将它们称为任务而不是线程
- ❖ 线程创建是通过clone（）系统调用完成的
- ❖ clone（）允许子任务共享父任务（进程）的地址空间
 - 标记控件行为

flag	meaning
CLONE_FS	File-system information is shared.
CLONE_VM	The same memory space is shared.
CLONE_SIGHAND	Signal handlers are shared.
CLONE_FILES	The set of open files is shared.

- ❖ 结构task_struct指向流程数据结构（共享或唯一）



中山大學
SUN YAT-SEN UNIVERSITY

计算机学院（软件学院）

SCHOOL OF COMPUTER SCIENCE AND ENGINEERING



谢谢